

Graph Engineering for Multi-Agentic Systems: The Andrew Ng Playbook

Based on Andrew Ng's courses, presentations, and the DeepLearning.AI curriculum
Including the July 2026 Agentic Knowledge Graphs course (DeepLearning.AI + Neo4j + Google ADK)
Independently compiled, July 2026 — not affiliated with or endorsed by Andrew Ng or DeepLearning.AI

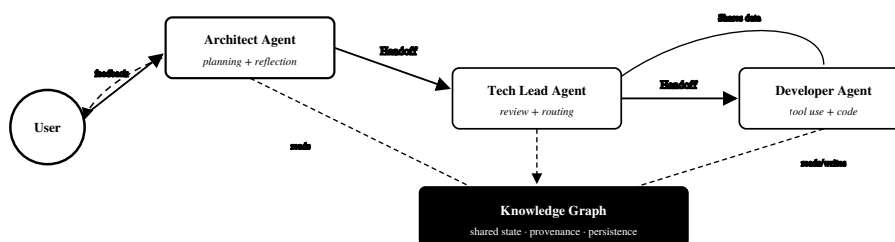


Fig. 1. Graph-grounded multi-agent architecture. The User delegates to an Architect Agent, which hands off to a Tech Lead and Developer Agent through typed handoffs. All agents read from and write to a shared Knowledge Graph. In-loop feedback flows back to the User at each stage.

Abstract—Andrew Ng proved that GPT-3.5 wrapped in an agentic workflow scores 95.1% on HumanEval, outperforming GPT-4 at 67.0% in zero-shot mode. The architecture matters more than the model. Ng identifies four design patterns — Reflection, Tool Use, Planning, and Multi-Agent Collaboration. Anthropic independently formalized five production workflows. This note presents all nine patterns with implementation guidance, maps how they compose from simple loops into graph architectures, and provides a staged build path from day one to production. In July 2026, Ng released a course on building agentic knowledge graphs from scratch, positioning graphs as the natural evolution beyond loops for systems that need persistent state, cross-session reasoning, and multi-agent coordination. The central thesis: a loop externalizes revision, a chain externalizes task order, a network externalizes role specialization, and a graph externalizes shared state and relationships. Once multiple workers must preserve facts across sessions, coordinate without copying transcripts, and explain why a result changed, the system needs a durable information layer.

Index Terms—Agentic AI, design patterns, reflection, tool use, planning, multi-agent collaboration, workflow orchestration, graph architecture, knowledge graphs, LLM agents.

I. INTRODUCTION

Asking an LLM to generate a useful output in one pass is difficult for the same reason that writing a polished essay in one uninterrupted pass is difficult. Ng's analogy is deliberately ordinary: a human writer drafts, reads, deletes, checks sources, reorganizes paragraphs, and asks other people for

criticism. A zero-shot LLM is usually denied all of those opportunities. Despite that constraint, modern models perform remarkably well. The central claim of agentic design is not that direct prompting is useless. The claim is that many tasks become substantially more reliable when the model is placed inside a workflow that permits iteration, evidence gathering, decomposition, and role separation.

The HumanEval example makes the architecture argument concrete. As reported by Ng, GPT-3.5 used zero-shot solves 48.1% of the benchmark, while GPT-4 used zero-shot solves 67.0%. When GPT-3.5 is wrapped in an agentic workflow, the reported score rises to 95.1%. The improvement from GPT-3.5 to GPT-4 is therefore dwarfed by the improvement associated with iterative workflow design. Ng summarizes: "AI agentic workflows will drive massive AI progress this year — perhaps even more than the next generation of foundation models." The statement shifts attention from model selection to system design.

Ng's four patterns provide the initial vocabulary. Reflection lets a model inspect and revise its work. Tool Use lets it obtain information outside its parameters. Planning lets it select steps. Multi-Agent Collaboration lets multiple instances contribute distinct capabilities. The key synthesis of this note is that these patterns are not isolated recipes. They are stages in the externalization of cognition. A loop

externalizes revision. A chain externalizes task order. A network externalizes role specialization. A graph externalizes shared state and relationships.

A. Why "Graphs vs. Loops" Matters Now

In July 2026, the agentic AI community is actively debating the next layer beyond loops. Peter Steinberger's post — "Are we still talking loops, or did we shift to graphs yet?" — drew over 2.7 million views. Within the same week, Ng released a free course on building agentic knowledge graphs from scratch (DeepLearning.AI, taught by Neo4j's Andreas Kollegger), covering graph construction, multi-agent system architecture on graphs, and practical demonstrations using Google's Agent Development Kit (ADK). The course positions graphs as the foundational infrastructure for agents that need to persist facts, share state, and reason across sessions.

The decision tree is simpler than the debate makes it sound. A single well-scoped task, retryable, with human review of output — a loop is correct. Multiple specialized agents, branching logic, state that must persist across sessions — graph-based orchestration. The two are not competing paradigms; they are successive stages in the externalization of cognition, and most production systems will use both.

B. Contributions

This synthesis makes three contributions. First, it presents Ng's four design patterns and Anthropic's five workflow patterns in one implementation-oriented account. Second, it maps how the patterns compose into loop, chain, network, and graph architectures. Third, it offers a decision framework and a staged build path that treats evaluation, cost, latency, and debuggability as first-class constraints.

II. THE FOUR DESIGN PATTERNS

A. Step 1: Reflection

Reflection is an iterative process in which an LLM examines an output, compares it with a task or rubric, identifies defects, and produces a revision. The pattern can be implemented with one model instance alternating between generator and critic prompts, or with two role-separated instances. The important property is not the number of model objects — it is the presence of an explicit feedback loop and a stopping rule.

```
@dataclass
class Critique:
    satisfactory: bool
    issues: list[str]
    revision_instructions: list[str]

def reflect(task, llm, max_iterations=3):
    draft = llm.generate(task)
    for _ in range(max_iterations):
        feedback = llm.evaluate(task=task, draft=draft)
        if feedback.satisfactory:
            return draft
        draft = llm.revise(task, draft, feedback)
    return draft
```

Ng's coding example is direct: a coder agent writes code, then receives a critique prompt. The evaluator often catches bugs, missing edge cases, or poor structure that the first pass overlooked. Ng reports: "I've been delighted by how much it improved my applications' results" and describes reflection as "pretty robust technology — I can almost always get them to work well."

Failure modes: (1) Self-confirmation — the critic repeats assumptions that produced the draft. (2) Rubric drift — the evaluator rewards fluency instead of correctness. (3) Non-monotonic revision — a fix introduces another bug. Controls: keep an immutable task statement, compare every revision against the rubric, run deterministic checks before subjective evaluation, cap iterations.

The first practical tip is to separate critique from rewriting. Do not ask "improve this" and accept an opaque replacement. Request a structured list of issues first, then feed that list into a distinct revision step. The second tip is to make the evaluator cite evidence from the draft, tests, or source material. A critique that says "the function might not handle edge cases" is less useful than one that says "line 12 does not handle the case where the input list is empty, which violates requirement 3 in the specification." The third tip is to store every intermediate artifact — draft, critique, revision, and stopping decision — so that engineers can diagnose whether failures came from weak generation, weak evaluation, or a faulty stopping rule. Without this record, debugging a reflection loop is like debugging a program without logs.

B. Step 2: Tool Use

Tool Use allows an LLM to select and call external capabilities: web search, code execution, databases, APIs. The model contributes language understanding and decision making; the tool contributes grounded data or deterministic action. Ng observes that early tool-use work came from the computer vision community because language models could not dir-

ectly manipulate images — "the only option was that the LLM generate a function call." A model that can execute code can verify its own logic. A model that can search can ground its claims. Tool use transforms an LLM from a closed system into one that can check its work against reality.

```
@dataclass
class ToolCall:
    name: str
    arguments: dict

tools = {
    "web_search": search_web,
    "run_code": execute_python,
    "query_db": database_query,
}

def agent_with_tools(task, llm, tools, max_steps=10):
    context = [task]
    for _ in range(max_steps):
        action = llm.choose_action(context, tools)
        if action.name == "finish":
            return llm.synthesize(context)
        result = tools[action.name](**action.arguments)
        context.append({"tool": action.name,
                       "result": result})
    return llm.synthesize(context)
```

Failure modes: (1) Wrong tool selection — calling a search when a database query is needed. (2) Invalid arguments — passing malformed inputs that the tool cannot process. (3) Trusting tool output blindly — not validating that a search result is relevant or that code execution produced the expected output type. (4) Tool overuse — calling tools when sufficient context already exists in the window, wasting tokens and latency. Controls: typed tool schemas with validated arguments, result confirmation before incorporation, permission boundaries that separate read from write access, and retry limits that prevent infinite tool-call loops.

C. Step 3: Planning

Planning uses an LLM to autonomously decide what sequence of steps to execute. Ng describes a live demo where his research agent's web search API returned a rate-limiting error: "To my surprise, the agent pivoted deftly to a Wikipedia search tool — which I had forgotten I'd given it — and completed the task." Ng's maturity assessment is explicit: "more emerging — sometimes my mind is blown by how well they work, but at least at this moment in time, I don't feel like I can always get them to work reliably." Planning agents therefore need tighter constraints: structured plan formats, dependency validation, bounded step counts, and fallback policies.

```
def plan_and_execute(objective, llm, tools, budget):
    plan = llm.create_plan(objective) # JSON steps
    results = []
    for step in plan.steps:
        result = execute_step(step, tools, results)
        results.append(result)
        if result.failed:
            plan = llm.replan(
                objective, results,
                remaining_budget=budget)
    return llm.synthesize(results)
```

The code illustrates the critical pattern: when a step fails, the agent replans *with context of what already worked*. This is the "preserve successful work across replans" principle. Without it, a failure at step 5 of 8 discards the successful results of steps 1–4 and starts over, wasting both tokens and time. With it, the replan builds on established ground.

Ng also mentions using research agents for his own work: "one piece of research that I don't feel like Googling myself — I send it to the research agent, come back in a few minutes and see what it's come up with." This captures the operational reality of planning agents: they are most useful when the human's time is more expensive than the agent's tokens, and when the task is well-defined enough that the agent's plan can be evaluated against objective criteria. Open-ended research ("find something interesting") produces plans that are hard to evaluate and easy to waste tokens on; focused research ("find all papers citing X that report Y") produces plans that are straightforward to validate.

Failure modes: (1) Over-planning — generating elaborate plans for simple tasks that a single call could handle. (2) Plan-execution gap — the plan describes steps the agent cannot actually perform with its available tools. (3) Cascading failure — one bad step corrupts all subsequent steps because they depend on its output. (4) Unbounded replanning — the agent replans indefinitely without converging on a solution. Controls: validate dependencies before execution (can every step's inputs be produced by a prior step or a tool?), preserve successful work across replans, bound total steps, and define explicit failure criteria ("if step fails twice after replan, escalate to human rather than trying a third time").

D. Step 4: Multi-Agent Collaboration

Multi-Agent Collaboration involves multiple LLM instances, each prompted with different roles, working together on tasks that benefit from specialization. Ng's ChatDev example: "completely open source, runs on my laptop." The system prompts one LLM to act as CEO, another as designer,

another as product manager, another as tester. "This flock of agents collaborate — if you tell it 'please develop a Go game,' they'll actually spend a few minutes writing code, testing it, iterating, and then generate surprisingly complex programs."

Ng calls multi-agent collaboration "more emerging" but says it "works much better than you might think." The practical tip: define an artifact contract for every handoff. The researcher returns claims with sources. The planner returns typed steps. The coder returns code and assumptions. The evaluator returns defects and a decision. Agents should communicate through artifacts and shared state, not unlimited conversational history. This constraint prepares the system for graph architecture.

Multi-agent debate — having different agents argue different positions — "actually results in better performance as well." The mechanism is straightforward: when two agents with different role prompts examine the same evidence, they tend to catch different error classes. A coder agent optimizes for functionality; a reviewer agent optimizes for edge cases; a tester agent optimizes for coverage. No single rubric captures all three — but three agents with three rubrics approximate a team. The practical limitation is cost: three agents consume three times the tokens, and if all three see the same evidence and optimize the same weak rubric, the system reproduces the same error three times at higher cost. The test for whether to add an agent: does this role catch an error class that existing roles demonstrably miss?

TABLE I
PATTERN MATURITY ASSESSMENT (ANDREW NG)

Pattern	Maturity	Ng's Assessment
<i>Reflection</i>	Robust	"I can almost always get them to work well"
<i>Tool Use</i>	Robust	Widely deployed, well-understood
<i>Planning</i>	Emerging	"Less mature, less predictable"
<i>Multi-Agent</i>	Emerging	"Works much better than you might think"

III. ANTHROPIC'S FIVE WORKFLOW PATTERNS

Anthropic's "Building Effective Agents" distinguishes *workflows* (LLMs and tools operating through predefined code paths) from *agents* (the model dynamically directs its own process). The distinction resists unnecessary autonomy: many reliable systems can be built from simple, composable workflows.

Prompt Chaining sends output of one call into the next in a fixed sequence; each stage performs a narrow transformation with programmatic checks between. Use when tasks

"decompose easily and cleanly into fixed subtasks." Strength: predictability, each stage testable separately. Weakness: inflexibility to unexpected inputs. A practical implementation uses typed intermediate outputs between stages so that each stage validates its input before processing — a malformed output from stage 2 is caught at stage 3's entrance, not at the final output.

Routing classifies an input and sends it to a specialized prompt, tool set, or workflow. A support system may route billing questions to a billing agent, technical incidents to a diagnostic workflow. Routing enables separation of concerns without requiring agents to converse. Main failure mode: misclassification. Controls include confidence thresholds (below 0.8 → fallback route) and explicit fallback paths for inputs that match no category. Routing is also the cheapest form of "intelligence" in a multi-agent system: a fast, small model classifies; a capable, expensive model processes.

Parallelization runs multiple calls simultaneously — sectioning (independent subtasks divided among workers) or voting (same task run multiple times, aggregator selects best). Sectioning reduces latency; voting increases robustness. The critical caveat: ten outputs from the same model with nearly identical prompts are not ten independent judgments. For voting to add genuine signal, the prompts or models should differ in ways that induce different error distributions — a code review with three different rubrics (correctness, security, performance) produces more signal than three identical reviews.

Orchestrator-Workers uses a central LLM to analyze a task, create subtasks dynamically, delegate them to workers, and synthesize outputs. This pattern combines Planning and Multi-Agent Collaboration. The architecture is powerful because it adapts, but the orchestrator can become a context bottleneck. Practical systems should require workers to return structured artifacts rather than raw conversation — bounded summaries with typed fields, not open-ended text. The orchestrator's context stays manageable when it receives a 200-token artifact per worker rather than a 5,000-token conversation transcript.

Evaluator-Optimizer separates generation from evaluation: one LLM produces a response, another evaluates against explicit criteria, the loop continues until a threshold is met. This is Reflection formalized as a production workflow with explicit role separation. Especially effective for translation, code with security requirements, professional communications where tone matters, and structured documents. Anthropic notes this pattern typically runs 2–4 cycles,

significantly improving output quality while maintaining accuracy.

A. Choosing Workflow Before Agent

The production default should be the simplest pattern that satisfies the task. Use a direct LLM call for a simple question. Use a chain when the steps are fixed. Use routing when requests belong to clear categories. Use parallel workers when independent coverage matters. Use an orchestrator when decomposition must be discovered. Use evaluator-optimizer when quality can be judged and improved iteratively. This order is not conservative for its own sake. Each step adds cost, latency, nondeterminism, and failure modes. Agentic design is effective when complexity is added in response to observed errors, not when every available pattern is included from the beginning.

TABLE II
MAPPING NG'S PATTERNS TO ANTHROPIC'S WORKFLOWS

Ng Pattern	Anthropic Workflow	Key Addition
<i>Reflection</i>	Evaluator-Optimizer	Explicit role separation and stopping criteria
<i>Tool Use</i>	Augmented LLM	Building block for all workflows
<i>Planning</i>	Orch.-Workers + Chaining	Dynamic vs. fixed decomposition
<i>Multi-Agent</i>	Parallel. + Orch.-Workers	Production orchestration

IV. FROM LOOPS TO GRAPHS

The combined pattern vocabulary describes a progression in how state, control, and responsibility are organized. The progression is not a maturity ladder every project must complete; many tasks should remain direct calls or fixed chains.

A. The Loop Stage

One agent repeatedly inspects and revises its own work. Reflection is the defining mechanism, often supported by tools. The loop is compact, easy to prototype, and effective for tasks within one context window. It breaks down when the task contains too much evidence, too many independent concerns, or too much history. Every iteration may resend task, draft, critique, tool results, and prior decisions — context becomes expensive and noisy. The loop's strength is also its limitation: everything lives in one context window, which means everything must fit in one context window. When it doesn't, the options are to truncate (losing informa-

tion), summarize (losing precision), or restructure — which leads to the chain.

B. The Chain Stage

Work is divided into a fixed sequence of specialized transformations. The chain externalizes order into application code. Each stage can have its own prompt, model, tools, and validation. Compared with a reflective loop, the chain is more predictable because the path is known. The limitation: unexpected cases must be anticipated. A chain that processes documents through "extract → validate → summarize → format" works perfectly when every document fits the expected structure. When a document is in an unexpected format, the chain either fails at the extraction stage (good — the failure is visible and early) or passes garbled output forward (bad — the failure compounds silently through every subsequent stage). Programmatic gates between stages are the fix: each gate checks whether the output of the previous stage meets the input contract of the next, and routes failures to a fallback rather than letting them propagate.

C. The Network Stage

Multiple role-specialized workers are introduced. An orchestrator delegates, workers act, results return for synthesis. The central limitation is context management. As Anthropic notes: "context grows too complex for one agent to manage effectively, creating performance bottlenecks." The orchestrator may become a conversational hub receiving every worker's full output. The context management problem scales quadratically with team size: with 5 workers, the orchestrator must hold 5 outputs; with 10 workers, 10 outputs; and each output may itself be the product of a multi-turn interaction. At some point the orchestrator's window is more summary than substance, and the summaries have lost the detail that matters. This is the moment that motivates the graph.

D. The Graph Stage

The graph stage externalizes shared state into a durable, queryable structure (Fig. 1). Each worker reads the subgraph relevant to its task and writes new entities and relations back. The orchestrator's context stays small; the shared state lives in the graph. This is the architecture Ng's July 2026 course teaches students to build from scratch.

The course, taught by Neo4j's Andreas Kollegger and hosted on DeepLearning.AI, covers four progressions: (1) what agentic knowledge graphs are and why agents need

them; (2) how to build the first agentic graph from raw data — entities as nodes, relationships as typed edges; (3) how multi-agent systems are architected on graph structures, with extraction agents feeding validated entities into a persistent store; and (4) building working implementations using Google's Agent Development Kit (ADK). The course positions knowledge graphs not as an advanced feature to add later but as a foundational component that should be designed from the beginning for any system requiring cross-session reasoning.

A knowledge graph serves multi-agent systems in three distinct roles. As *shared memory* for orchestrator-workers: workers read from and write to the graph directly, replacing the fragile alternative of passing summaries through the orchestrator's bottleneck. This is the multi-agent analogue of Anthropic's principle that "the session is not the context window." As *grounding layer* for evaluator-optimizer: the evaluator checks claims against graph edges with provenance, producing feedback like "triple (X, works_at, Y) does not exist in the graph; the graph contains (X, left, Y) from document Z" rather than "this seems off." As *persistent world model* for loops: the graph survives context-window flushes — the agent forgets, the graph does not.

E. A Minimal Graph Schema

A useful schema begins with five node types: Entity (person, organization, product, concept), Claim (statement that may be supported or contradicted), Source (document, API response, test result), Artifact (plan, draft, code, report), and Run (execution record). Edges capture relationships: `mentions`, `supports`, `contradicts`, `derived_from`, `supersedes`. Every claim retains provenance. Every revision points to the prior version. The minimal write operation should be additive: rather than overwriting a claim silently, create a new version and link it with `supersedes`.

F. Graph Architecture Is Not Automatic Truth

A graph can preserve errors as efficiently as facts. Entity resolution can merge distinct organizations. Extraction can attach the wrong date. A confident evaluator can mark a weak source as sufficient. Graph-grounded systems therefore require data quality mechanisms: schema validation, canonical identifiers, provenance, conflict representation, confidence calibration, and periodic review.

V. THE BENCHMARK EVIDENCE

TABLE III
HUMAN EVAL CODING BENCHMARK (REPORTED BY NG)

Configuration	Score (%)
GPT-3.5 zero-shot	48.1
GPT-4 zero-shot	67.0
GPT-3.5 + agentic workflow	95.1

Key insight: "the improvement from GPT-3.5 to GPT-4 is dwarfed by incorporating an iterative agent workflow." This means: invest in workflow architecture, not just model upgrades. Ng: "if you're looking forward to running your thing on GPT-5 zero-shot, you may be able to get closer to that level of performance on some applications with agentic reasoning on an earlier model."

One additional insight Ng emphasizes: fast token generation is important for agentic workflows because the LLM generates tokens for other LLMs to read, not for humans. "Being able to generate tokens way faster than any human can read is fantastic." Generating more tokens quickly from even a slightly lower quality LLM "might give good results compared to slower tokens from a better LLM — because it may let you go around this loop a lot more times." This makes fast inference a direct multiplier for agentic system performance.

The benchmark data has a second implication that is less discussed but equally important. If a weaker model with a strong process can outperform a stronger model asked to answer once, then *model selection and workflow design are not independent decisions*. The optimal system is not necessarily the strongest model in the simplest workflow; it may be a weaker, cheaper, faster model in a compound workflow that uses the savings to run more iterations, more workers, or more evaluation rounds. This is the economic argument for the staged build path: each stage's cost must be justified by measurable lift, and a cheaper model that funds more iterations may outperform an expensive model that runs once.

Ng's insight about fast inference compounds with the staged build path. At the reflection stage, fast inference lets the loop run more iterations within the same latency budget. At the multi-agent stage, fast inference lets the orchestrator dispatch and receive results from workers faster, reducing end-to-end latency. At the graph stage, fast inference lets extraction agents process larger corpora in less time, building richer graphs that benefit all downstream agents. The speed of the model is not just a cost optimization — it is a

capability multiplier that enables compound architectures that would be impractical with slow inference.

VI. PRACTICAL IMPLEMENTATION GUIDE

The recommended build path starts with the smallest reliable loop and adds one capability at a time. Each stage should have a baseline, a task-level metric, a cost budget, and a rollback path.

A. Start with Reflection — Day 1

Choose one existing LLM call whose output can be evaluated. Add a second call that critiques the first against an explicit rubric. Feed the critique into a revision call. Compare the revised output with the baseline. Expected improvement: 10–30% quality boost, but the team should measure its own lift. The implementation is deliberately minimal: two additional LLM calls and a for-loop. The rubric should be written before the first run, not discovered after — "improve this" is not a rubric. A good starting rubric names the specific qualities the output must have (correct information, appropriate length, required sections) and the specific defects the critic should look for (unsupported claims, missing citations, logical contradictions). The rubric itself is a kind of unit test for the output.

B. Add Tool Use — Day 2

Give the agent access to at least one tool: code execution or web search. Let the agent decide when to use it. Validate tool results before incorporating them. Watch for the "rerouting around failures" moment Ng describes — the agent discovering and using a fallback tool is one of the clearest signs that the agentic architecture is adding value beyond what a fixed script could provide. Start with read-only tools before adding write access; a tool that reads a database is safe to experiment with, while a tool that modifies production data requires the same permission controls you would apply to a junior engineer.

C. Build a Planning Agent — Week 1

For complex tasks, have the LLM write a plan as structured JSON before executing. Execute each step, collect results. If a step fails, replan with context of what already worked. Preserve successful work across replans. The practical constraint: planning agents consume significantly more tokens than direct calls because the plan itself is generated text, each step's execution generates text, and the synthesis of results generates text. Budget accordingly — a planning

agent that runs 8 steps at 2,000 tokens each plus a 3,000-token synthesis is consuming ~19,000 tokens, roughly 10× a direct call. The lift in task completion rate must justify this cost, and for simple tasks it usually does not.

D. Go Multi-Agent — Week 2

Split your task into roles: generator + critic minimum. Use same model, different system prompts. Iterate until critic is satisfied or iteration limit reached. Define artifact contracts for every handoff.

E. Wire Into a Graph — Month 1

Add persistent state between agent runs. Start simple: shared JSON file or database. Graduate to knowledge graph when agents need to chain facts across sessions. Add provenance tracking and version history. This is the stage Ng's July 2026 course covers in detail: building the graph from scratch using Google ADK, with extraction agents feeding entities into a Neo4j-backed knowledge graph that multiple agents query and extend.

TABLE IV
IMPLEMENTATION TIMELINE

Step	Time	Complexity	Expected Lift
<i>Add reflection</i>	1 day	Low	10–30% quality
<i>Add tool use</i>	1 day	Low	New capabilities
<i>Planning agent</i>	1 week	Medium	Complex tasks
<i>Multi-agent</i>	2 weeks	Medium	Better quality
<i>Graph architecture</i>	1 month	High	Persistent, scalable

VII. DECISION FRAMEWORK

TABLE V
WHEN TO USE EACH PATTERN

Situation	Start With	Why
Better output quality	<i>Reflection</i>	Cheapest, most reliable improvement
Need external data	<i>Tool Use</i>	Grounds answers in real information
Complex multi-step	<i>Planning</i>	Decomposes into manageable steps
Multiple perspectives	<i>Multi-Agent</i>	Different roles catch different errors
Cross-session state	<i>Graph arch.</i>	Survives context window flushes
Simple single QA	<i>Zero-shot</i>	Don't over-engineer

The rule of thumb: if the current pattern's failure mode is understood and the next pattern addresses that specific failure, add it. If the failure mode is unclear, measure before

adding complexity. Each pattern adds cost, latency, and debugging surface. The cheapest pattern that satisfies the task is usually the right one.

A. Five Decision Rules

Five rules formalize the decision framework for practitioners. *Rule 1: Start with the cheapest pattern.* A reflection loop is cheaper than a multi-agent system, and a multi-agent system is cheaper than a graph architecture. Start with the cheapest and add complexity only when a specific, measured failure demands it. *Rule 2: Measure before promoting.* Before adding the next pattern, establish a baseline with the current pattern and measure the failure rate you expect the new pattern to address. If the failure rate is below 5%, the new pattern's complexity cost likely exceeds its benefit. *Rule 3: Match control to risk.* High-stakes tasks (financial transactions, safety-critical operations) should use predictable patterns (chains, evaluation loops) with explicit gates. Low-stakes tasks (brainstorming, research drafts) can tolerate the unpredictability of planning and multi-agent systems. *Rule 4: Count tokens, not agents.* The cost of an agentic system is proportional to tokens consumed, not to the number of conceptual agents. A three-agent system where each agent runs for 20,000 tokens costs the same as one agent running for 60,000 tokens. Design for token efficiency, not conceptual elegance. *Rule 5: The graph earns itself.* A knowledge graph is justified when the same entity or relationship is queried by more than one agent or across more than one session. A graph that is written to once and never queried is a database table with extra overhead.

B. Anti-Patterns to Avoid

Seven anti-patterns recur in production agentic systems. *The everything-agent:* one agent with every tool, every role, and a 50-page system prompt that tries to handle all cases — this agent has no clear responsibility and no clear failure mode, making it impossible to debug. *The echo chamber:* multiple agents with identical prompts and identical evidence producing identical outputs at higher cost — more agents does not mean more intelligence. *The infinite loop:* a reflection or planning agent without a stopping rule that iterates until the token budget is exhausted, producing marginal stylistic changes rather than substantive improvements. *The phantom graph:* a knowledge graph with an elaborate ontology that no agent ever queries — infrastructure cost without value. *The conversational bottleneck:* an orchestrator that receives every worker's full conversation transcript, growing

its context linearly with team size until it exceeds the window. *The missing baseline:* deploying an agentic system without first measuring the performance of a zero-shot call, so the team cannot tell whether the agentic overhead is producing lift or just cost. *The premature agent:* building a multi-agent system for a task that a single well-prompted call handles perfectly, motivated by the architectural pattern rather than the task's requirements.

VIII. THE GRAPHS VS. LOOPS DEBATE

The July 2026 debate crystallized around a real question: when does a loop stop being sufficient and a graph become necessary? The answer maps onto three criteria. First, *session persistence:* if the work spans multiple sessions and transcript replay is no longer practical, state needs to live outside the context window — the graph. Second, *cross-agent coordination:* if multiple specialized agents must share facts without copying entire transcripts through a bottleneck, the graph is the shared blackboard. Third, *traceability:* if the system must explain why a result changed, provenance on typed edges is the infrastructure.

Loops handle the first criterion poorly (state lives in-context and is flushed), the second not at all (a loop is one agent), and the third weakly (the conversation history is the only record). Graphs handle all three structurally. But loops are simpler to build, cheaper to run, and sufficient for the vast majority of single-agent, single-session tasks. The engineering decision is not "which is better" but "when does the loop's limitation become the binding constraint."

The practical answer from the community's experience: start with a loop, measure where it fails, and add graph infrastructure at the specific failure point — not before. A loop with a state file is already partway to a graph. A graph that nobody queries is an overengineered loop.

The new DeepLearning.AI course makes the graph stage accessible. Previously, building a knowledge graph required expertise in graph databases, ontology design, and NLP extraction pipelines. The course shows that with modern tools (Google ADK for agent orchestration, Neo4j for graph storage, and an LLM for extraction), a working agentic knowledge graph can be built in under an hour. This dramatically lowers the bar for teams considering the transition from loops to graphs, while the decision framework above ensures they make the transition at the right time.

A. The Compound Architecture

In practice, production systems rarely use one pattern in isolation. The most effective architectures compound patterns: a planning agent that uses reflection at each step, tool use for evidence gathering, multi-agent collaboration for review, and graph architecture for persistence. The compound effect is where the real performance lives — not in any single pattern, but in the specific combination tuned to the task. The build path described in Section VI is designed to make this compounding incremental and measurable rather than all-at-once and opaque.

Ng’s insight about fast token generation is especially relevant for compound architectures. In a system where a planning agent generates a plan, a coder agent implements each step, a reviewer agent critiques each implementation, and a graph stores the results, the total token consumption can be 10–50× a single direct call. Fast inference at acceptable quality — “generating more tokens quickly from even a slightly lower quality LLM might give good results compared to slower tokens from a better LLM” — makes these compound systems economically viable rather than theoretically interesting.

APPENDIX B

Production Readiness Checklist

TABLE VII
PRODUCTION READINESS CHECKLIST BY STAGE

Stage	Element	Ask Yourself
Reflection	Rubric	Is the evaluation criterion explicit and stable?
	Iteration cap	Is there a maximum number of revision rounds?
	Artifact log	Is every draft, critique, and revision stored?
Tool Use	Schema	Are tool names, arguments, and return types validated?
	Permissions	Are read and write permissions separated?
	Fallback	What happens when the tool fails or rate-limits?
Planning	Plan format	Are plans structured JSON with dependencies?
	Step budget	Is total step count bounded?
	Replan policy	Does successful work survive replanning?
Multi-Agent	Artifact contracts	Does every handoff use typed schemas?

Stage	Element	Ask Yourself
Graph Arch.	Role differentiation	Do roles catch genuinely different error classes?
	Orchestrator budget	Does the orchestrator receive bounded summaries?
	Provenance	Does every edge trace to a source document?
	Versioning	Are overwrites replaced by supersession links?
	Entity resolution	Are resolution decisions inspectable?

APPENDIX C

Worked Example: From Zero-Shot to Graph in Five Steps

Consider a team building a code-review assistant. The progression follows the staged build path exactly.

Day 0 — Zero-shot baseline: The team sends code to the LLM with the prompt “review this code for bugs.” The output is a free-form paragraph that sometimes catches real issues and sometimes praises correct code. Measured accuracy: 55%.

Day 1 — Add reflection: The team adds a second call: “Here is your review. Does it cite specific line numbers? Does it distinguish severity levels? Revise.” The revised review is measurably better: specific line numbers, severity labels, fewer false positives. Measured accuracy: 72%.

Day 2 — Add tool use: The reviewer agent can now execute the code against a test suite and read linter output. Reviews cite concrete test failures instead of hypothetical bugs. Measured accuracy: 84%.

Week 1 — Add planning: For large PRs (>500 lines), the agent writes a plan: “review security-sensitive files first, then business logic, then tests.” Each section gets a focused review with the right rubric. Measured accuracy on large PRs: 79% (up from 61% with reflection alone).

Week 2 — Add multi-agent: A security reviewer (system prompt: “you are a security auditor, assume every input is malicious”) runs alongside the general reviewer. The security agent catches injection vulnerabilities the general agent missed. Combined accuracy: 88%.

Month 1 — Add graph: Review findings are stored as entities (vulnerability, code pattern, affected file) with typed relations (found_in, similar_to, fixed_by). When a new PR touches a file that previously had a vulnerability, the reviewer agent queries the graph and flags the pattern. The graph accumulates institutional knowledge across hundreds of PRs — knowledge that no single context window could hold. Measured accuracy on repeat patterns: 95%.

The progression illustrates two principles. First, each stage earns the right to the next by addressing a specific, measured failure of the previous stage. Second, the compound effect — 55% → 72% → 84% → 88% → 95% — is far greater than any single stage's contribution. The architecture, not the model, is doing the heavy lifting.

IX. LIMITATIONS

Three limitations are worth stating plainly. First, Ng himself calls Planning and Multi-Agent "emerging" — they do not always work. "At least at this moment in time, I don't feel like I can always get them to work reliably." Second, agentic workflows trade latency for quality. Ng: "we need to learn to delegate tasks to an AI agent and patiently wait minutes, maybe even hours, for a response." Third, each loop iteration costs tokens. More agents do not automatically produce more intelligence — if all agents see the same evidence and optimize the same weak rubric, the system reproduces the same error several times at higher cost.

Additionally, the HumanEval benchmark is a coding benchmark. The 95.1% figure should not be assumed to transfer directly to all domains. Teams should build their own evaluation sets and measure lift in their specific context. Graph architecture adds its own failure modes: entity resolution errors, stale provenance, schema drift, and the operational cost of maintaining a durable store. The graph amplifies corpus quality, just as the loop amplifies the judgment of its builder.

X. INDUSTRY ADOPTION

The adoption trajectory gives context for the maturity assessments. A 2025 industry poll found 93% of IT leaders plan to introduce autonomous AI agents within two years, and nearly half have already started pilot implementations. Deloitte predicts that by 2025, one-quarter of companies using generative AI will have launched pilot agentic AI projects, growing to 50% by 2027. In his year-end letter for 2025, Ng characterized the year as "the dawn of the AI industrial era" — a period when AI moved from research and experimentation into industrial-scale infrastructure, with capital expenditure across the industry exceeding \$300 billion.

The adoption pattern mirrors the staged build path of this note. Most organizations start with reflection (adding a review step to an existing LLM call), then add tool use (connecting the model to databases and APIs), and only then

consider planning and multi-agent architectures. Graph architecture is the least adopted stage, which is consistent with its position at the end of the build path and with the fact that Ng's course on the topic is only days old. The debate on X — "are we still talking loops, or did we shift to graphs yet?" — reflects the moment at which the early-adopter segment is crossing from network-stage to graph-stage, while the majority is still in the loop-stage or chain-stage.

The practical implication for teams: do not skip stages. Reflection is the highest-ROI pattern for almost any existing LLM application, and it is also the most mature. A team that cannot demonstrate measurable lift from reflection is unlikely to benefit from the complexity of multi-agent coordination or graph architecture. The build path is not a ladder of prestige; it is an engineering sequence where each stage earns the right to the next by demonstrating that the previous stage's failure mode is the binding constraint.

XI. CONCLUSION

Ng's four patterns are the vocabulary. Anthropic's five workflows are the grammar. The graph is the language. The single most important takeaway: workflow architecture matters more than model capability. GPT-3.5 with an agentic workflow (95.1%) outperforms GPT-4 in zero-shot mode (67.0%).

The build path is incremental. Start with reflection — it works today, reliably, on most tasks where quality can be evaluated. Add tools to ground answers in real data. Build planning agents for complex tasks. Go multi-agent when specialization adds measurable signal. Wire into a graph when state must persist across sessions and agents must share a world model.

The compound effect of layering patterns is where the real performance lives. Each pattern addresses a specific limitation of the previous stage: reflection addresses single-pass errors, tools address knowledge gaps, planning addresses complexity, multi-agent addresses perspective limits, and graphs address memory limits. The progression is not mandatory, but it is directional.

Ng's observation captures the broader significance: "The path to AGI feels like a journey rather than a destination, but this type of agent workflow could help us take a small step forward on this very long journey." The engineering version of the same insight: every important output should be traceable to a task, a plan, an artifact, a source, an evaluator decision, and a bounded execution record. When that is true,

loops, tools, plans, workers, and graphs become composable engineering mechanisms rather than opaque behavior. The path from loops to graphs is not a path from simplicity to complexity — it is a path from implicit state to explicit state, from volatile memory to durable memory, and from estimation to evidence.

A reliable agentic system should make the following sentence true: *every important output can be traced to a task, a plan, an artifact, a source, an evaluator decision, and a bounded execution record*. When that sentence is false, more autonomy usually increases uncertainty. When it is true, the system is composable and debuggable regardless of how many patterns it compounds. The question to ask of any agentic system is not "how many patterns does it use" but "can I explain, for any output, why it is what it is." The graph — with its typed edges, provenance, and version history — is the infrastructure that makes the answer yes.

ACKNOWLEDGMENT & SOURCES

This document is an independent synthesis for study. It is not affiliated with or endorsed by Andrew Ng, DeepLearning.AI, Anthropic, Neo4j, or Google. **Andrew Ng**, "What's Next for AI Agentic Workflows," Sequoia Capital AI Ascent, Mar. 2024; "Four AI Agent Strategies," The Batch, DeepLearning.AI, Mar. 2024; "Agentic Design Patterns Parts 2–4," The Batch, Apr. 2024; "Agentic AI," DeepLearning.AI course, Oct. 2025; Agentic Knowledge Graphs course (DeepLearning.AI + Neo4j + Google ADK), Jul. 2026. **E. Schluntz & B. Zhang**, "Building Effective Agents," Anthropic Engineering, Dec. 2024. **Anthropic**, "Building Effective AI Agents: Architecture Patterns and Implementation Frameworks," 2026. Referenced papers: Self-Refine (Madaan et al., 2023), Reflexion (Shinn et al., 2023), CRITIC (Gou et al., 2024), Toolformer (Schick et al., 2023), HuggingGPT (Shen et al., 2023), ChemCrow (Bran et al., 2023), AutoGen (Wu et al., 2023), ChatDev (Qian et al., 2024). Industry data: 93% of IT leaders plan AI agents within 2 years (2025 Connectivity Benchmark Report); Deloitte predicts 25% of companies will have agentic pilots by 2025. All diagrams are original to this document.

APPENDIX: GLOSSARY

TABLE VI
TERMS USED IN THIS NOTE

Term	Meaning
<i>Zero-shot</i>	Single LLM call, no iteration.
<i>Agentic workflow</i>	LLM prompted multiple times with iteration.
<i>Reflection</i>	LLM critiques and revises its output.
<i>Tool use</i>	LLM calls external capabilities.
<i>Planning</i>	LLM decomposes task into steps.
<i>Multi-agent</i>	Multiple LLM instances collaborate.
<i>Prompt chaining</i>	Fixed sequence of LLM calls.

Term	Meaning
<i>Routing</i>	Classify input, send to specialist.
<i>Parallelization</i>	Multiple LLM calls simultaneously.
<i>Orchestrator-workers</i>	Central LLM delegates to workers.
<i>Evaluator-optimizer</i>	One LLM generates, another evaluates.
<i>Graph architecture</i>	Shared persistent memory with typed edges.
<i>Knowledge graph</i>	Entities as nodes, relationships as edges, with provenance.
<i>Artifact contract</i>	Typed schema for agent handoffs.
<i>Provenance</i>	Source from which a fact was derived.

This document is an independent synthesis assembled for study. It is not a publication of, and is not affiliated with or endorsed by, Andrew Ng, DeepLearning.AI, Anthropic, Neo4j, or Google. All benchmark numbers are from Andrew Ng's published analysis. All quotes are from his public talks and writings. Diagrams are original.